

Explique de forma extramamente completa e didática, com exemplos que podem ser aplicados em produção do banco de dados MySQL versão 8.0.46, exemplos de inserção, deleção e seleção filtro e backup lógico e físico incluindo ferramentas com exemplos práticos

⊗ Thought for 4 seconds ∨

O usuário pede uma explicação extremamente completa e didática sobre MySQL 8.0.46, cobrindo exemplos de INSERT, DELETE, SELECT com filtros, e backup lógico e físico com ferramentas e exemplos práticos. Preciso fornecer uma resposta detalhada. Para isso, vou pesquisar informações sobre MySQL 8.0.46, comandos DML, backup lógico (mysqldump, mysqlpump) e backup físico (MySQL Enterprise Backup, Percona XtraBackup). Vou buscar fontes oficiais e tutoriais.

🔍 Found 69 web pages 🔗 🌐 📄

Os resultados da pesquisa fornecem alguns exemplos, mas preciso de mais detalhes. Vou abrir alguns links relevantes para obter informações mais completas.

📖 Read 5 pages MySQL :: MySQL 8.0 Reference Manual :: 6.5.4 mysqldump ↗

9.3.1 Establishing a Backup Policy - MySQL 8.0 Reference Manual Including MySQL NDB Cluster 8.0 ↗

MySQL :: MySQL 8.0 Reference Manual :: 6.5.6 mysqlpump ↗

MySQL Enterprise Backup User's Guide (Version 8.0.35) ↗

How to Use Percona XtraBackup for MySQL Hot Backups - How to Use Percona XtraBackup for My... ↗

Esses recursos fornecem muitas informações sobre backup. Agora preciso estruturar a resposta. Vou organizar em seções: introdução, exemplos de INSERT, DELETE, SELECT com filtros, backup lógico (mysqldump, mysqlpump, MySQL Shell), backup físico (MySQL Enterprise Backup, Percona XtraBackup), e considerações finais. Vou incluir exemplos práticos. A resposta será extensa e didática.

Índice

1. Preparação do Ambiente
2. Inserção de Dados (INSERT)
3. Seleção com Filtros (SELECT + WHERE)
4. Exclusão de Dados (DELETE)
5. Backup Lógico (Logical Backup)
6. Backup Físico (Physical Backup)
7. Boas Práticas e Recomendações

1. Preparação do Ambiente

Todos os exemplos foram testados no **MySQL 8.0.46**. Vamos criar uma base de dados e uma tabela para servirem de laboratório:

```
sql
```

```
-- Criar o banco de dados
```

```
CREATE DATABASE IF NOT EXISTS empresa;
```

```
USE empresa;
```

```
-- Criar uma tabela de funcionários
```

```
CREATE TABLE IF NOT EXISTS funcionarios (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nome VARCHAR(100) NOT NULL,  
    cargo VARCHAR(50) NOT NULL,  
    salario DECIMAL(10,2) NOT NULL,  
    data_contratacao DATE NOT NULL,  
    ativo BOOLEAN DEFAULT TRUE,  
    email VARCHAR(100) UNIQUE,  
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- Criar uma tabela de departamentos para exemplos com JOIN
```

```
CREATE TABLE IF NOT EXISTS departamentos (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nome VARCHAR(50) NOT NULL UNIQUE,  
    orcamento DECIMAL(15,2) NOT NULL  
);
```

```
-- Adicionar chave estrangeira na tabela funcionarios
```

```
ALTER TABLE funcionarios ADD COLUMN departamento_id INT;  
ALTER TABLE funcionarios  
    ADD CONSTRAINT fk_departamento  
    FOREIGN KEY (departamento_id) REFERENCES departamentos(id);
```

2. Inserção de Dados (INSERT)

2.1. Inserção de uma única linha

```
sql
```

```
INSERT INTO departamentos (nome, orcamento)
VALUES ('Tecnologia', 100000.00);
```

sql

```
INSERT INTO funcionarios (nome, cargo, salario, data_contratacao, email, departamento_id)
VALUES ('Ana Silva', 'Desenvolvedora Sênior', 8500.00, '2023-01-15', 'ana.silva@empresa.com', 1);
```

2.2. Inserção de múltiplas linhas de uma vez

sql

```
INSERT INTO funcionarios (nome, cargo, salario, data_contratacao, email, departamento_id) VALUES
('Carlos Souza', 'Analista de Dados', 6200.00, '2023-03-10', 'carlos.souza@empresa.com', 1),
('Mariana Oliveira', 'Gerente de Projetos', 12000.00, '2022-06-01', 'mariana.oliveira@empresa.com', 1),
('João Pereira', 'Desenvolvedor Júnior', 4500.00, '2024-01-20', 'joao.pereira@empresa.com', 1),
('Fernanda Lima', 'Designer UX', 5800.00, '2023-08-05', 'fernanda.lima@empresa.com', 1),
('Roberto Alves', 'DevOps', 7200.00, '2023-11-01', 'roberto.alves@empresa.com', 1);
```

sql

```
-- Múltiplos departamentos
INSERT INTO departamentos (nome, orcamento) VALUES
('Marketing', 75000.00),
('RH', 50000.00),
('Financeiro', 90000.00);
```

2.3. Inserção com SELECT (copiando dados de outra tabela)

sql

```
-- Criar uma tabela de backup de funcionários
CREATE TABLE funcionarios_backup LIKE funcionarios;

-- Copiar dados de funcionarios para funcionarios_backup
```

```
INSERT INTO funcionarios_backup (nome, cargo, salario, data_contratacao, ativo, email, departamento_id)
SELECT nome, cargo, salario, data_contratacao, ativo, email, departamento_id
FROM funcionarios
WHERE salario > 5000;
```

2.4. Inserção com ON DUPLICATE KEY UPDATE (UPSERT)

sql

```
-- Se o email já existir, atualiza o cargo e o salário
INSERT INTO funcionarios (nome, cargo, salario, data_contratacao, email, departamento_id)
VALUES ('Ana Silva', 'Arquiteta de Software', 9500.00, '2023-01-15', 'ana.silva@empresa.com', 1)
ON DUPLICATE KEY UPDATE
    cargo = VALUES(cargo),
    salario = VALUES(salario);
```

2.5. Obtendo o ID da última inserção

sql

```
INSERT INTO funcionarios (nome, cargo, salario, data_contratacao, email, departamento_id)
VALUES ('Paulo Mendes', 'Estagiário', 2500.00, '2024-02-01', 'paulo.mendes@empresa.com', 1);

SELECT LAST_INSERT_ID(); -- Retorna o ID gerado automaticamente
```

3. Seleção com Filtros (SELECT + WHERE)

3.1. Filtros básicos (comparadores)

sql

```
-- Igualdade
SELECT * FROM funcionarios WHERE cargo = 'Desenvolvedor Sênior';

-- Diferente
```

```
SELECT * FROM funcionarios WHERE cargo != 'Gerente de Projetos';
```

```
-- Maior que / Menor que
```

```
SELECT * FROM funcionarios WHERE salario > 7000;
```

```
SELECT * FROM funcionarios WHERE salario < 5000;
```

```
-- Maior ou igual / Menor ou igual
```

```
SELECT * FROM funcionarios WHERE salario >= 6000;
```

3.2. Múltiplas condições (AND / OR)

```
sql
```

```
-- AND: todas as condições devem ser verdadeiras
```

```
SELECT * FROM funcionarios
```

```
WHERE salario > 5000 AND cargo LIKE '%Desenvolvedor%';
```

```
-- OR: pelo menos uma condição deve ser verdadeira
```

```
SELECT * FROM funcionarios
```

```
WHERE cargo = 'Gerente de Projetos' OR salario > 10000;
```

```
-- Combinação de AND e OR (use parênteses para clareza)
```

```
SELECT * FROM funcionarios
```

```
WHERE (cargo = 'Desenvolvedor Sênior' OR cargo = 'DevOps')
```

```
AND salario > 7000;
```

3.3. Filtro por lista (IN)

```
sql
```

```
SELECT * FROM funcionarios
```

```
WHERE cargo IN ('Desenvolvedor Sênior', 'DevOps', 'Analista de Dados');
```

```
-- Equivalente com OR
```

```
SELECT * FROM funcionarios
```

```
WHERE cargo = 'Desenvolvedor Sênior'
```

```
OR cargo = 'DevOps'
```

```
OR cargo = 'Analista de Dados';
```

3.4. Filtro por intervalo (BETWEEN)

```
sql
```

```
-- Salário entre 5000 e 8000
SELECT * FROM funcionarios
WHERE salario BETWEEN 5000 AND 8000;

-- Datas entre dois valores
SELECT * FROM funcionarios
WHERE data_contratacao BETWEEN '2023-01-01' AND '2023-12-31';
```

3.5. Filtro com padrões (LIKE)

sql

```
-- Começa com "Desenvolvedor"
SELECT * FROM funcionarios WHERE cargo LIKE 'Desenvolvedor%';

-- Termina com "or"
SELECT * FROM funcionarios WHERE cargo LIKE '%or';

-- Contém "Silva" em qualquer posição
SELECT * FROM funcionarios WHERE nome LIKE '%Silva%';

-- Coringas: % (qualquer sequência) e _ (um único caractere)
SELECT * FROM funcionarios WHERE nome LIKE 'A_a%'; -- Ana, Ada, etc.
```

3.6. Filtro com valores nulos (IS NULL / IS NOT NULL)

sql

```
-- Funcionários sem email
SELECT * FROM funcionarios WHERE email IS NULL;

-- Funcionários com email cadastrado
SELECT * FROM funcionarios WHERE email IS NOT NULL;
```

3.7. Filtros com data e hora

sql

```
-- Funcionários contratados no primeiro semestre de 2023
SELECT * FROM funcionarios
WHERE data_contratacao >= '2023-01-01' AND data_contratacao <= '2023-06-30';
```

```
-- Usando funções de data
SELECT * FROM funcionarios
WHERE YEAR(data_contratacao) = 2023 AND MONTH(data_contratacao) <= 6;

-- Funcionários contratados nos últimos 30 dias
SELECT * FROM funcionarios
WHERE data_contratacao >= CURDATE() - INTERVAL 30 DAY;
```

3.8. Ordenação (ORDER BY) e Limite (LIMIT)

sql

```
-- Ordenar por salário (crescente)
SELECT * FROM funcionarios ORDER BY salario ASC;

-- Ordenar por salário (decrescente)
SELECT * FROM funcionarios ORDER BY salario DESC;

-- Ordenação múltipla
SELECT * FROM funcionarios ORDER BY departamento_id, salario DESC;

-- Limitar resultados
SELECT * FROM funcionarios ORDER BY salario DESC LIMIT 3; -- Top 3 salários

-- Paginação: pular 2 e pegar 3
SELECT * FROM funcionarios ORDER BY id LIMIT 3 OFFSET 2;

-- ou
SELECT * FROM funcionarios ORDER BY id LIMIT 2, 3;
```

3.9. Filtros com JOIN

sql

```
-- Inner Join: funcionários com seus departamentos
SELECT f.nome, f.cargo, f.salario, d.nome AS departamento
FROM funcionarios f
INNER JOIN departamentos d ON f.departamento_id = d.id;

-- Left Join: todos os funcionários, mesmo sem departamento
SELECT f.nome, f.cargo, COALESCE(d.nome, 'Sem departamento') AS departamento
FROM funcionarios f
LEFT JOIN departamentos d ON f.departamento_id = d.id;
```

3.10. Filtros com subconsultas

sql

```
-- Funcionários com salário acima da média
SELECT * FROM funcionarios
WHERE salario > (SELECT AVG(salario) FROM funcionarios);

-- Funcionários do departamento com maior orçamento
SELECT f.*
FROM funcionarios f
WHERE f.departamento_id = (
    SELECT id FROM departamentos
    ORDER BY orcamento DESC
    LIMIT 1
);
```

3.11. Filtros com funções agregadas (HAVING)

sql

```
-- Departamentos com mais de 2 funcionários
SELECT d.nome, COUNT(f.id) AS total_funcionarios
FROM departamentos d
LEFT JOIN funcionarios f ON d.id = f.departamento_id
GROUP BY d.id, d.nome
HAVING total_funcionarios > 2;

-- Departamentos com salário médio acima de 7000
SELECT d.nome, AVG(f.salario) AS media_salarial
FROM departamentos d
JOIN funcionarios f ON d.id = f.departamento_id
GROUP BY d.id, d.nome
HAVING media_salarial > 7000;
```

4. Exclusão de Dados (DELETE)

4.1. DELETE com filtro (recomendado)

sql


```
-- Excluir um funcionário específico
DELETE FROM funcionarios WHERE id = 1;

-- Excluir funcionários com salário abaixo de 3000
DELETE FROM funcionarios WHERE salario < 3000;

-- Excluir funcionários inativos
DELETE FROM funcionarios WHERE ativo = FALSE;
```

4.2. DELETE com múltiplas condições

sql

```
-- Excluir funcionários de um departamento específico contratados antes de 2023
DELETE FROM funcionarios
WHERE departamento_id = 1 AND data_contratacao < '2023-01-01';
```

4.3. DELETE com subconsulta

sql

```
-- Excluir funcionários de departamentos com orçamento abaixo de 50000
DELETE FROM funcionarios
WHERE departamento_id IN (
    SELECT id FROM departamentos WHERE orcamento < 50000
);
```

4.4. DELETE com LIMIT (para evitar bloqueios longos)

sql

```
-- Excluir em lotes de 100 registros
DELETE FROM funcionarios WHERE ativo = FALSE LIMIT 100;
-- Execute repetidamente até não haver mais registros
```

4.5. DELETE com JOIN (MySQL suporta exclusão em múltiplas tabelas)

sql

```
-- Excluir funcionários e seus registros relacionados em outra tabela
DELETE f, fb
FROM funcionarios f
JOIN funcionarios_backup fb ON f.id = fb.id
WHERE f.ativo = FALSE;
```

4.6. TRUNCATE (exclusão rápida de todos os dados)

sql

```
-- Remove TODOS os registros e reseta o AUTO_INCREMENT
TRUNCATE TABLE funcionarios_backup;

-- Diferença: DELETE sem WHERE remove todos, mas mantém o AUTO_INCREMENT
DELETE FROM funcionarios_backup; -- Mais lento e mantém a sequência
```

4.7. DELETE com segurança (usando transações)

sql

```
-- Iniciar transação
START TRANSACTION;

-- Verificar quais registros serão afetados
SELECT * FROM funcionarios WHERE salario < 3000;

-- Executar o DELETE
DELETE FROM funcionarios WHERE salario < 3000;

-- Verificar o resultado
SELECT ROW_COUNT() AS registros_afetados;

-- Se estiver tudo certo, confirma
COMMIT;

-- Se algo deu errado, desfaz
-- ROLLBACK;
```

Cuidados essenciais com DELETE

Cuidado	Explicação	Exemplo
Sempre use WHERE	DELETE sem WHERE remove TODOS os registros	✗ DELETE FROM funcionarios;
Teste com SELECT primeiro	Veja o que será excluído antes de deletar	✓ SELECT * FROM funcionarios WHERE ...
Use transações	Permite desfazer em caso de erro	START TRANSACTION; ... COMMIT;
Considere FOREIGN KEY	Pode falhar se houver registros filhos	Verifique dependências antes
DELETE é logado	Cada exclusão é registrada no binlog (impacto em performance)	Use LIMIT para grandes volumes

5. Backup Lógico (Logical Backup)

Backup lógico consiste em exportar dados e estrutura como **comandos SQL** que podem ser executados para recriar o banco. É portátil entre versões e sistemas operacionais.

5.1. mysqldump (ferramenta padrão)

O `mysqldump` é a ferramenta nativa do MySQL para backups lógicos ²³. Produz um arquivo com instruções `CREATE TABLE` e `INSERT` que recriam o banco ²².

Backup completo de todos os bancos

```
bash

mysqldump -u root -p --all-databases --single-transaction --master-data=2 > /backup/full_backup_$(date +%Y%m%d).sql
```

- `--single-transaction` : gera um backup consistente sem bloquear tabelas InnoDB ²²
- `--master-data=2` : inclui a posição do binlog (comentado) para replicação ²²

Backup de um banco específico

```
bash
```

```
mysqldump -u root -p --databases empresa > /backup/empresa_$(date +%Y%m%d).sql
```

Backup de tabelas específicas

```
bash
```

```
mysqldump -u root -p empresa funcionarios departamentos > /backup/tabelas_$(date +%Y%m%d).sql
```

Backup apenas da estrutura (sem dados)

```
bash
```

```
mysqldump -u root -p --no-data empresa > /backup/estrutura_empresa.sql
```

Backup apenas dos dados (sem estrutura)

```
bash
```

```
mysqldump -u root -p --no-create-info empresa > /backup/dados_empresa.sql
```

Backup com compressão em tempo real

```
bash
```

```
mysqldump -u root -p --single-transaction empresa | gzip > /backup/empresa_$(date +%Y%m%d).sql.gz
```

Backup com opções para consistência em replicação

```
bash
```

```
mysqldump -u root -p \  
  --all-databases \  
  --single-transaction \  
  --triggers \  
  --routines \  
  --events \  
  --master-data=2 \  
  --flush-logs \  
  > /backup/full_backup_$(date +%Y%m%d_%H%M).sql
```

Restaurando um backup mysqldump

```
bash
```

```
# Restaurar banco de dados
```

```
mysql -u root -p empresa < /backup/empresa_20260101.sql
```

```
# Restaurar com compressão
```

```
gunzip -c /backup/empresa_20260101.sql.gz | mysql -u root -p empresa
```

```
# Restaurar todos os bancos
```

```
mysql -u root -p < /backup/full_backup_20260101.sql
```

5.2. mysqlpump (paralelização - deprecated)

O `mysqlpump` foi introduzido para oferecer **backup paralelo**, mas é **deprecated** desde o **MySQL 8.0.34** e será removido em versões futuras ³³. A Oracle recomenda usar `mysqldump` ou o MySQL Shell ³³.

```
bash
```

```
# Exemplo de mysqlpump (não recomendado para uso futuro)
```

```
mysqlpump -u root -p --databases empresa --single-transaction > /backup/empresa.sql
```

5.3. MySQL Shell Dump Utilities (recomendado)

O MySQL Shell oferece utilitários modernos com **paralelismo**, **compressão** e **progresso** em tempo real ²³ ³³:

```
bash
```

```
# Backup de uma instância inteira
```

```
mysqlsh -- util dump-instance --all --outputUrl=/backup/instance_dump
```

```
# Backup de um schema específico
```

```
mysqlsh -- util dump-schema empresa --outputUrl=/backup/empresa_dump
```

```
# Com compressão e paralelismo
```

```
mysqlsh -- util dump-schema empresa \  
  --outputUrl=/backup/empresa_dump \  
  --threads=4 \  
  --compress=true
```

```
# Restaurar
mysqlsh -- util load-dump /backup/empresa_dump
```

5.4. Backup de tabelas em formato CSV

```
sql
```

```
-- Exportar para CSV
SELECT nome, cargo, salario
INTO OUTFILE '/tmp/funcionarios.csv'
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
FROM funcionarios;

-- Importar de CSV
LOAD DATA INFILE '/tmp/funcionarios.csv'
INTO TABLE funcionarios_import
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
(nome, cargo, salario);
```

6. Backup Físico (Physical Backup)

Backup físico copia os **arquivos do banco de dados** no sistema de arquivos. É mais rápido para restauração, especialmente em bancos grandes, e preserva índices e estruturas internas ⁴⁴.

6.1. MySQL Enterprise Backup (MEB) - Solução Comercial

O MySQL Enterprise Backup é a solução oficial da Oracle para backup físico ⁴⁴. Suporta backup **hot** (com o banco em execução) e é **obrigatório** para tabelas com tablespace criptografado ⁴⁴.

Backup completo (single-file)

```
bash
```

```
mysqlbackup --user=root --password=senha \
--backup-image=/backup/full_backup.mbi \
```

```
--backup-dir=/backup \  
--show-progress \  
backup-to-image
```



Backup completo para diretório

bash

```
mysqlbackup --user=root --password=senha \  
--backup-dir=/backup/$(date +%Y%m%d) \  
--with-timestamp \  
backup
```

Backup com compressão

bash

```
mysqlbackup --user=root --password=senha \  
--backup-image=/backup/full_backup_compressed.mbi \  
--compress \  
backup-to-image
```



Backup incremental

bash

```
# Baseado no último backup completo  
mysqlbackup --user=root --password=senha \  
--backup-dir=/backup/incremental_$(date +%Y%m%d) \  
--incremental \  
--incremental-base=dir:/backup/full_20260101 \  
backup
```



Restaurar

bash

```
# Parar o MySQL primeiro  
mysqlbackup --backup-image=/backup/full_backup.mbi \  

```

```
--backup-dir=/backup/restore \  
image-to-backup-dir
```

Depois copiar os arquivos para o datadir

6.2. Percona XtraBackup - Solução Open Source

O Percona XtraBackup (PXB) é uma ferramenta **gratuita e open source** que faz backup físico **hot** (sem bloquear o banco) ⁵⁶ .

⚠ Importante: O PXB 8.0 só funciona com MySQL 8.0 devido a mudanças no redo log e data dictionary ⁵⁷ .

Instalação

```
bash
```

Ubuntu/Debian

```
wget https://downloads.percona.com/downloads/Percona-XtraBackup-8.0/.../percona-xtrabac  
kup-80_8.0.32-26-1.focal_amd64.deb  
sudo dpkg -i percona-xtrabackup-80_8.0.32-26-1.focal_amd64.deb  
sudo apt-get install -f
```

RHEL/CentOS

```
sudo yum install -y https://repo.percona.com/yum/percona-release-latest.noarch.rpm  
sudo percona-release enable tools release  
sudo yum install -y percona-xtrabackup-80
```

⁵⁶

Criar usuário para backup

```
sql
```

```
CREATE USER 'xtrabackup'@'localhost' IDENTIFIED BY 'XtraPass123!';  
GRANT BACKUP_ADMIN, PROCESS, RELOAD, LOCK TABLES,  
      REPLICATION CLIENT, CREATE TABLESPACE,  
      SYSTEM_VARIABLES_ADMIN, SELECT ON *.*  
      TO 'xtrabackup'@'localhost';  
FLUSH PRIVILEGES;
```

⁵⁶

Backup completo


```
bash
```

```
# Realizar backup
xtrabackup --backup \
  --user=xtrabackup \
  --password=XtraPass123! \
  --target-dir=/backups/mysql/full_$(date +%Y%m%d)
```

56

O backup nesse momento **não é consistente**. É necessário prepará-lo:

```
bash
```

```
# Aplicar o redo log para tornar o backup consistente
xtrabackup --prepare --target-dir=/backups/mysql/full_20260101
```

56

Backup incremental

```
bash
```

```
# 1. Backup completo (base)
xtrabackup --backup \
  --user=xtrabackup --password=XtraPass123! \
  --target-dir=/backups/mysql/full_base

# 2. Primeiro incremental (baseado no full)
xtrabackup --backup \
  --user=xtrabackup --password=XtraPass123! \
  --target-dir=/backups/mysql/incr_day1 \
  --incremental-basedir=/backups/mysql/full_base

# 3. Segundo incremental (baseado no primeiro)
xtrabackup --backup \
  --user=xtrabackup --password=XtraPass123! \
  --target-dir=/backups/mysql/incr_day2 \
  --incremental-basedir=/backups/mysql/incr_day1
```

56

Preparar incremental para restauração

```
bash
```

```
# Preparar o full backup
xtrabackup --prepare --apply-log-only --target-dir=/backups/mysql/full_base

# Aplicar cada incremental na ordem
xtrabackup --prepare --apply-log-only \
  --target-dir=/backups/mysql/full_base \
  --incremental-dir=/backups/mysql/incr_day1

xtrabackup --prepare --apply-log-only \
  --target-dir=/backups/mysql/full_base \
  --incremental-dir=/backups/mysql/incr_day2

# Finalizar (sem --apply-log-only)
xtrabackup --prepare --target-dir=/backups/mysql/full_base
```

Restaurar

```
bash

# Parar o MySQL
sudo systemctl stop mysql

# Limpar o datadir (ou mover para backup)
sudo rm -rf /var/lib/mysql/*

# Copiar os arquivos de volta
xtrabackup --copy-back --target-dir=/backups/mysql/full_base

# Ajustar permissões
sudo chown -R mysql:mysql /var/lib/mysql

# Iniciar o MySQL
sudo systemctl start mysql
```



6.3. Backup físico manual (cold backup)

Se você pode **parar o MySQL**, um backup físico manual é simples:

```
bash

# Parar o MySQL
sudo systemctl stop mysql
```

```
# Copiar todo o diretório de dados
sudo cp -r /var/lib/mysql /backup/mysql_$(date +%Y%m%d)

# Ou com tar
sudo tar -czf /backup/mysql_$(date +%Y%m%d).tar.gz /var/lib/mysql

# Iniciar o MySQL novamente
sudo systemctl start mysql
```

- Vantagem:** Extremamente simples.
- Desvantagem:** Requer downtime.

7. Boas Práticas e Recomendações

7.1. Comparativo entre tipos de backup

Característica	Backup Lógico (mysqldump)	Backup Físico (XtraBackup/MEB)
Velocidade de backup	Lento em grandes bases	Rápido (copia arquivos)
Velocidade de restauração	Lenta (re-executa SQL)	Rápida (copia arquivos)
Portabilidade	Alta (qualquer versão/SO)	Baixa (versão específica)
Backup online	Sim (--single-transaction)	Sim (hot backup)
Backup incremental	Limitado	Suportado
Tamanho do backup	Geralmente menor	Geralmente maior
Complexidade	Baixa	Média/Alta

7.2. Estratégia de backup recomendada

```
bash
```

```
# Domingo: Backup completo (físico)
xtrabackup --backup --target-dir=/backups/full_$(date +%Y%m%d)

# Segunda a Sábado: Backup incremental
xtrabackup --backup --target-dir=/backups/incr_$(date +%Y%m%d) \
  --incremental-basedir=/backups/full_$(date +%Y%m%d -d "last sunday")

# Diariamente: Backup lógico (para portabilidade)
mysqldump --single-transaction --all-databases | gzip > /backups/logical_$(date +%Y%m%d).sql.gz

# A cada hora: Backup do binlog (point-in-time recovery)
mysqlbinlog --read-from-remote-server --host=localhost \
  --user=root --password=senha --raw \
  --stop-never binlog.000001 > /backups/binlog/
```

7.3. Verificação de backups

```
bash

# Verificar integridade de um dump SQL (tenta restaurar em um banco de teste)
mysql -u root -p test_db < /backup/backup.sql 2>&1 | grep -i error

# Verificar backup físico do XtraBackup
xtrabackup --prepare --target-dir=/backups/full_20260101

# Testar restauração completa em ambiente de homologação periodicamente
```

7.4. Limpeza de backups antigos

```
bash

# Manter últimos 7 dias de backups diários
find /backups -name "*.sql.gz" -mtime +7 -delete
find /backups -name "full_*" -mtime +30 -delete
find /backups -name "incr_*" -mtime +7 -delete
```

7.5. Segurança

```
bash
```

```
# Criptografar backups
mysqldump --all-databases | gzip | openssl enc -aes-256-cbc -salt \
    -out /backups/backup_$(date +%Y%m%d).sql.gz.enc -pass pass:chave_secreta

# Descriptografar para restaurar
openssl enc -d -aes-256-cbc -in /backups/backup.sql.gz.enc \
    -pass pass:chave_secreta | gunzip | mysql -u root -p
```

Resumo dos Comandos Mais Usados

Operação	Comando
INSERT único	INSERT INTO tabela (colunas) VALUES (valores);
INSERT múltiplo	INSERT INTO tabela (colunas) VALUES (v1), (v2),(v3);
SELECT com filtro	SELECT * FROM tabela WHERE condicao;
DELETE com filtro	DELETE FROM tabela WHERE condicao;
Backup lógico	mysqldump -u root -p --single-transaction banco > backup.sql
Restore lógico	mysql -u root -p banco < backup.sql
Backup físico (XtraBackup)	xtrabackup --backup --target-dir=/backup
Preparar backup físico	xtrabackup --prepare --target-dir=/backup
Restore físico	xtrabackup --copy-back --target-dir=/backup

Com essas ferramentas e exemplos, você tem um guia completo para gerenciar dados no MySQL 8.0.46 em produção, desde operações diárias de CRUD até estratégias robustas de backup e recuperação. Lembre-se sempre: **teste seus backups em ambiente de homologação periodicamente** — um backup só é útil se puder ser restaurado com sucesso!



69 web pages